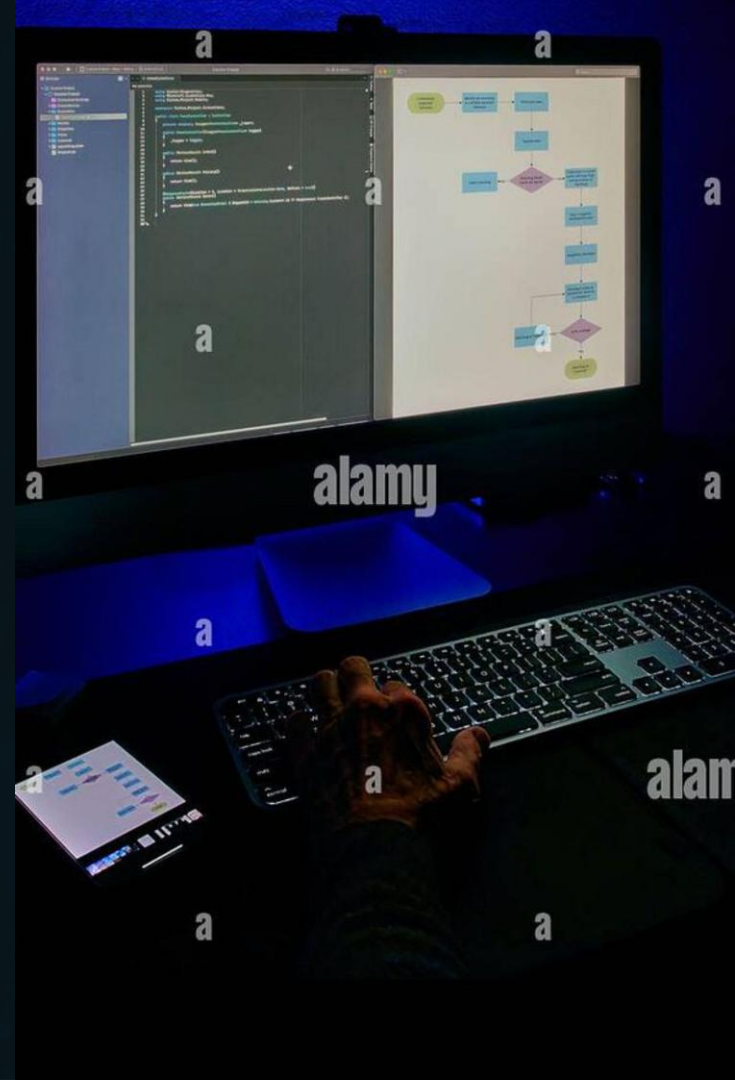


Konsep Dasar Algoritma & Pemrograman Terstruktur

Pondasi berpikir komputasional untuk setiap programmer — dari memahami logika algoritma hingga menuliskannya dalam notasi yang terstruktur dan mudah dipahami.

MATERI PEMROGRAMAN DASAR



Materi 1 — 10 Menit

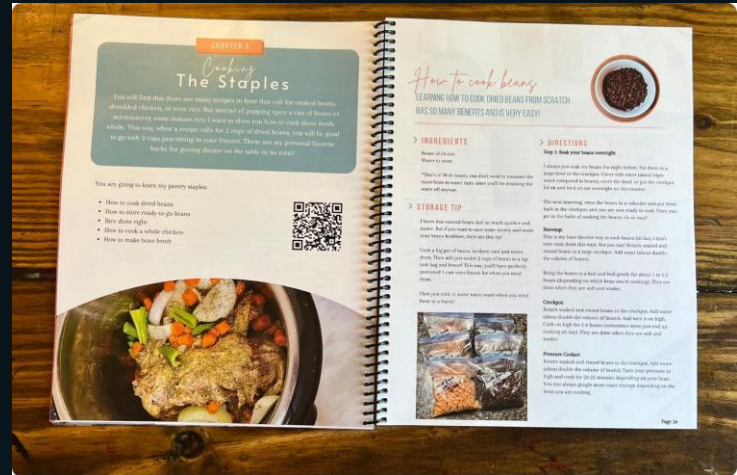
Apa Itu Algoritma?

Definisi

Algoritma adalah urutan langkah-langkah logis dan sistematis yang disusun untuk memecahkan suatu masalah atau mencapai tujuan tertentu.

Dalam pemrograman, algoritma ibarat "resep masakan" atau "denah perjalanan" — ia adalah rencana yang matang sebelum kita mulai menulis kode.

- ❑ Tanpa algoritma yang jelas, kode program akan sulit dirancang, rentan error, dan sulit diperbaiki.



5 Ciri Wajib Sebuah Algoritma

Tidak semua urutan langkah bisa disebut algoritma. Sebuah algoritma yang baik **harus memenuhi kelima kriteria** berikut ini:

1

Input

Memiliki nol atau lebih masukan data sebagai bahan pemrosesan.

2

Output

Menghasilkan minimal satu keluaran atau hasil yang bermakna.

3

Definiteness

Setiap langkah didefinisikan dengan **jelas dan tidak ambigu** — tidak ada makna ganda.

4

Finiteness

Algoritma **harus berhenti** setelah sejumlah langkah terbatas. Tidak boleh ada infinite loop tanpa syarat berhenti.

5

Effectiveness

Setiap langkah harus **sederhana dan dapat dieksekusi** secara manual sebagai operasi dasar.

Paradigma Pemrograman Terstruktur

Masalah: Era Spaghetti Code (1960-an)

Pada tahun 1960-an, programmer bebas menggunakan **statement GOTO** untuk melompat ke bagian kode mana saja. Hasilnya: alur program berantakan seperti mie, sulit dibaca, sulit di-debug, dan hampir mustahil di-maintain.

Solusi: Structured Program Theorem (1966)

Böhm dan Jacopini membuktikan secara matematis bahwa **setiap program yang dapat dihitung dapat ditulis hanya menggunakan tiga struktur kendali** — tanpa perlu GOTO sama sekali.

Ini menjadi landasan seluruh pemrograman modern yang kita kenal hari ini.



Era Spaghetti

Böhm &
Jacopini

Pemrograman
Terstruktur

Tiga Struktur Kendali Utama

Setiap program terstruktur dibangun dari kombinasi tiga blok dasar ini. Kuasai ketiganya, dan kamu bisa menulis program apa pun.



1. Sequential (Berurutan)

Instruksi dieksekusi **baris per baris**, **dari atas ke bawah** tanpa lompatan.
Analogi: prosedur check-in maskapai
— Antri → Serahkan tiket → Terima boarding pass.

Kata kunci: `INPUT`, `PROCESS`, `OUTPUT`



2. Selection (Pemilihan)

Eksekusi bergantung pada **kondisi Benar/Salah**. Analogi: "Jika hujan, bawa payung. Jika tidak, bawa topi."

Kata kunci: `IF`, `THEN`, `ELSE`, `SWITCH/CASE`

3. Iteration (Perulangan)

Eksekusi **berulang selama kondisi Benar**, berhenti saat kondisi Salah.
Analogi: "Makan nasi *sampai* kenyang."

Kata kunci: `WHILE`, `FOR`, `DO-WHILE`

```
{ border: 1px solid #ddd; }
```

```
> thead > tr > th, .table-bordered > thead > tr > td, .table-bordered > tbody > tr > td, .table-bordered > tbody > tr > td, .table-bordered > tfoot > tr > th, .table-bordered > tfoot > tr > td { }
```

```
> thead > tr > th, .table-bordered > thead > tr > td { border-bottom-width: 2px; }
```

```
> tbody > tr:nth-of-type(odd) { background-color: #f9f9f9; }
```

```
> tbody > tr:hover { background-color: #f5f5f5; }
```

```
.table-bordered {  
  border: 1px solid #ddd;  
}  
  
.table-bordered > thead > tr > th,  
.table-bordered > thead > tr > td,  
.table-bordered > tbody > tr > th,  
.table-bordered > tbody > tr > td,  
.table-bordered > tfoot > tr > th,  
.table-bordered > tfoot > tr > td {  
  border: 1px solid #ddd;  
}  
  
.table-bordered > thead > tr > th,  
.table-bordered > thead > tr > td {  
  border-bottom-width: 2px;  
}  
  
.table-striped > tbody > tr:nth-of-type(odd) {  
  background-color: #f9f9f9;  
}  
  
.table-hover > tbody > tr:hover {  
  background-color: #f5f5f5;  
}
```

Mengapa Harus Terstruktur?



Readability

Kode mudah dibaca alur logikanya secara **top-down**. Programmer lain bisa memahami kode tanpa kesulitan.



Maintainability

Mudah diperbaiki dan dimodifikasi tanpa merusak bagian lain dari program.



Debugging

Mudah melacak error karena alur **jelas dan terprediksi** — setiap blok memiliki Single Entry dan Single Exit.



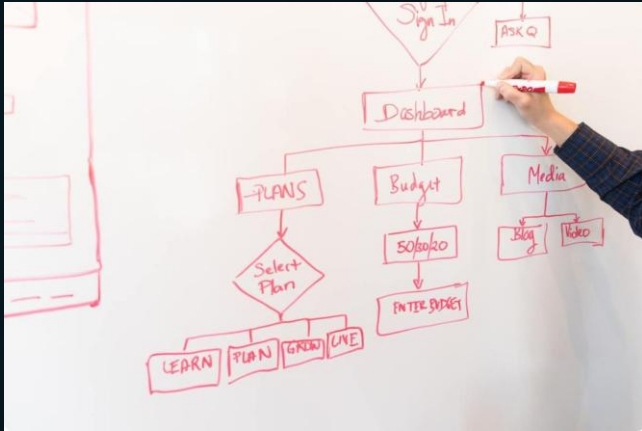
Reusability

Blok struktur (modul/fungsi) bisa **dipakai ulang** di bagian program lain atau di proyek berbeda.

Materi 3 — 25 Menit

Flowchart: Bagan Alir Algoritma

Flowchart adalah representasi visual algoritma menggunakan simbol standar ANSI/ISO. Wajib dipahami setiap programmer sebelum menulis kode.



Simbol-Simbol Wajib

Simbol	Nama	Fungsi
Oval	Terminal	Start / Stop
Persegi Panjang	Process	Perhitungan / Assignment
Belah Ketupat	Decision	Percabangan Ya/Tidak
Parallelogram	I/O	Input / Output data
Panah	Flowline	Arah alur eksekusi

- ❗ **Aturan Flowchart:** Alur utama atas→bawah, hanya satu Start dan satu End, Decision wajib punya 2 keluar (Ya/Tidak), hindari garis bersilang.

Pseudocode: Kode Palsu yang Terstruktur

Apa Itu Pseudocode?

Pseudocode adalah penulisan algoritma menggunakan **bahasa sehari-hari yang terstruktur** — mirip kode program, tapi **bebas dari sintaksis bahasa pemrograman spesifik**.

Tujuannya: fokus pada **logika**, bukan pada aturan bahasa pemrograman.

- Kata kunci ditulis **HURUF BESAR**: IF, THEN, ELSE, WHILE, FOR
- **Indentasi wajib** untuk blok di dalam struktur
- Variabel dideklarasikan di awal: `DECLARE integer nilai`
- Assignment menggunakan `<-` atau `=`

Contoh: Menghitung Luas Persegi Panjang

```
ALGORITHM LuasPersegiPanjang
DECLARE integer panjang, lebar, luas

START
  OUTPUT "Masukkan Panjang: "
  INPUT panjang
  OUTPUT "Masukkan Lebar: "
  INPUT lebar

  luas <- panjang * lebar

  OUTPUT "Luas = ", luas
END
```

Perbandingan: Flowchart vs Pseudocode

Kedua notasi ini saling melengkapi. Flowchart unggul secara visual, sementara Pseudocode lebih mudah dikonversi langsung ke kode program.

Flowchart

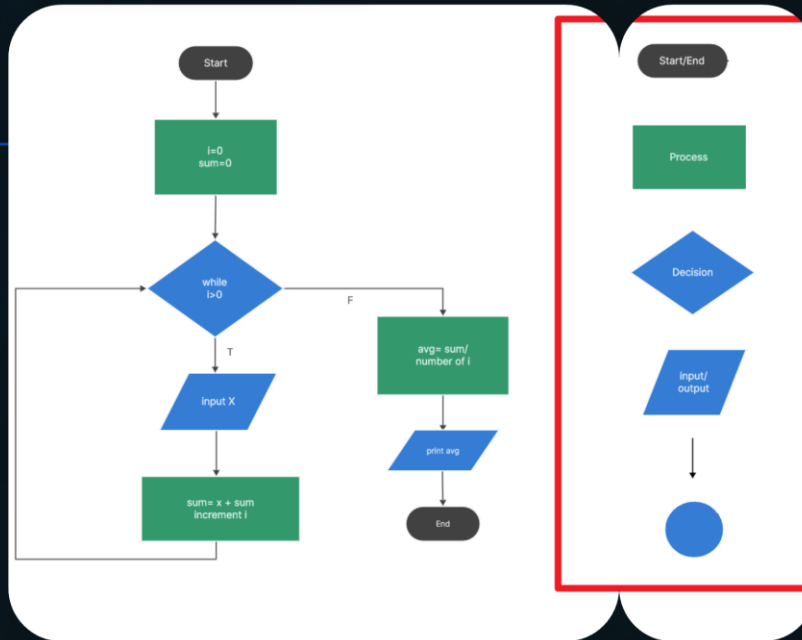
- Representasi grafis/visual
- Mudah dipahami oleh non-programmer
- Menggunakan simbol standar ANSI/ISO
- Cocok untuk mendokumentasikan alur kompleks
- Lebih sulit diedit secara cepat

Pseudocode

- Representasi tekstual terstruktur
- Mirip kode program asli
- Mudah dikonversi ke bahasa pemrograman apa pun
- Cocok untuk diskusi antar programmer
- Lebih cepat ditulis dan diedit

Flowchart

Visual, simbol grafis, mudah dipahami oleh semua orang



Pseudocode

Tekstual, mirip kode, mudah dikonversi ke bahasa pemrograman

Materi 4 — 10 Menit

Studi Kasus: Cek Kelulusan

Mari kita terapkan semua yang telah dipelajari! Buat algoritma untuk menginput nilai ujian dan menentukan status kelulusan.

Soal

Buat algoritma yang menginput nilai ujian. Jika nilai ≥ 75 , output "LULUS". Selain itu, output "TIDAK LULUS".

Panduan Flowchart

- Gunakan simbol **Parallelogram** untuk INPUT nilai
- Gunakan simbol **Belah Ketupat** untuk Decision: nilai ≥ 75 ?
- Dua cabang keluar: Ya $\rightarrow$ LULUS, Tidak $\rightarrow$ TIDAK LULUS

Panduan Pseudocode

```
DECLARE integer nilai
INPUT nilai
IF nilai  $\geq$  75 THEN
    OUTPUT "LULUS"
ELSE
    OUTPUT "TIDAK LULUS"
END IF
```



Kunci Belajar: Kuasai 5 ciri algoritma, 3 struktur kendali (Sequential, Selection, Iteration), dan dua notasi (Flowchart & Pseudocode) sebagai fondasi pemrogramanmu!

